

PROGRAMACIÓN IV

Trabajo Práctico 4

OBJETIVO GENERAL

Desarrollar un nuevo módulo Proveedores sobre el proyecto base u1_ej_8_integrador aplicando los conceptos de Path, Query, Body, Pydantic, manejo de errores y arquitectura modular en FastAPI. El estudiante implementará el CRUD completo de Proveedores siguiendo el mismo patrón de los módulos existentes (Categoría y Producto), utilizando almacenamiento en memoria y validación con Pydantic.

INSTRUCCIÓN AL SISTEMA

El sistema a construir es una API REST de gestión de catálogo que actualmente administra Categorías y Productos. El alumno debe extenderla agregando un nuevo módulo Proveedores que permita registrar y gestionar los proveedores que abastecen los productos del catálogo.

El módulo Proveedores debe implementarse respetando la misma arquitectura modular del proyecto base: separación clara entre router y service, modelos Pydantic correctamente definidos, códigos HTTP adecuados y manejo de errores con HTTPException. Los datos se almacenan en memoria (listas) durante la ejecución del servidor, sin persistencia externa.

El sistema se considerará cumplido cuando un administrador pueda gestionar el ciclo de vida completo de los proveedores (crear, listar, consultar, actualizar y desactivar) a través de la API, y todas las validaciones y reglas de negocio se cumplan.

MARCO TEÓRICO

Concepto	Aplicación en el proyecto
FastAPI	Framework moderno de Python para crear APIs RESTful con validación automática de datos y documentación interactiva.
Pydantic	Biblioteca de validación de datos que define schemas para request/response bodies con validación automática de tipos y constraints.
Arquitectura modular	Patrón que organiza el código en módulos independientes (categoria/, producto/, proveedor/), cada uno con sus propios archivos router.py, schemas.py y services.py.
Path Parameters	Parámetros dinámicos en la URL (ej: /proveedores/{id}) que identifican un recurso específico.
Query Parameters	Parámetros opcionales en la URL (ej: ?skip=0&limit=10) para filtrar o paginar resultados.

HTTPException

Mecanismo de FastAPI para devolver errores HTTP con código y mensaje específico (404, 409, 422).

CASO PRÁCTICO

Paso 1 – Probar el sistema base

1. Clonar el repositorio base

https://github.com/profcarlosamartinez/fastapi_backend/tree/main/u1_ej_8_integrador

2. Instalar dependencias:

```
pip install -r requirements.txt
```

3. Ejecutar el servidor:

```
fastapi dev app/main.py
```

4. Probar los endpoints existentes en Swagger (<http://localhost:8000/docs>) y con REST Client.

5. Comprender el flujo: main → router (endpoints) → services (lógica) → schemas (validación).

Paso 2 – Estructura esperada del proyecto

La estructura del proyecto debe quedar de la siguiente manera:

```
u1_ej_8_integrador/
├── app/
│   ├── __init__.py
│   └── main.py
├── modules/
│   ├── __init__.py
│   ├── categoria/
│   │   ├── __init__.py
│   │   ├── routers.py
│   │   ├── schemas.py
│   │   └── services.py
│   ├── producto/
│   │   ├── __init__.py
│   │   ├── routers.py
│   │   ├── schemas.py
│   │   └── services.py
│   └── proveedor/ ← NUEVO MÓDULO
│       ├── __init__.py
│       ├── routers.py
│       ├── schemas.py
│       └── services.py
├── tests/           # Archivos .http de prueba
├── requirements.txt
└── README.md
```

Paso 3 – Crear el módulo Proveedores

a) Definir los schemas en `app/modules/proveedor/schemas.py`

Crear los modelos Pydantic con los siguientes campos:

Campo	Tipo	Requerido	Default	Validación
codigo	str	Sí	—	min_length=1
razon_social	str	Sí	—	min_length=3
cuit	str	Sí	—	min_length=11 , max_length=15
email	str	No	""	—
telefono	str	No	""	—
activo	bool	No	True	—

El schema `ProveedorBase` contendrá todos los campos anteriores. `ProveedorCreate` hereda de `ProveedorBase` y no agrega campos. `ProveedorRead` agrega el campo `id` generado por el backend. `ProveedorUpdate` tiene todos los campos como `Optional` para actualización parcial.

b) Implementar los endpoints en `app/modules/proveedor/routers.py`

Crear un `APIRouter` con `prefix="/proveedores"` y `tags=["Proveedores"]` que exponga:

Método	Endpoint	Código	Descripción
POST	<code>/proveedores/</code>	201	Crear un nuevo proveedor
GET	<code>/proveedores/</code>	200	Listar proveedores (con paginación y filtros)
GET	<code>/proveedores/{id}</code>	200	Obtener detalle de un proveedor por ID
PUT	<code>/proveedores/{id}</code>	200	Actualizar todos los campos de un proveedor (reemplazo total)

PUT	/proveedores/{id}/desactivar	200	Desactivar (borrado lógico) un proveedor
-----	------------------------------	-----	--

Parámetros de consulta para GET /proveedores/:

Parámetro	Tipo	Default	Descripción
skip	int	0	Cantidad de registros a saltar (ge=0)
limit	int	10	Cantidad máxima a retornar (ge=1, le=50)
activo	Optional[bool]	None	Filtrar por estado (True=activos, False=inactivos, None=todos)

c) Implementar la lógica de negocio en `app/modules/proveedor/services.py`

Crear las funciones que operan sobre una lista en memoria (mismo patrón que los servicios de Categoría y Producto). Incluir las reglas de negocio definidas en la sección correspondiente: validar código único (RN-02), no desactivar ya desactivado (RN-05), etc.

d) Registrar el router en `main.py`

Importar el router de proveedores y registrarlo en la aplicación:

```
from app.modules.proveedor.routers import router as proveedor_router
```

```
app.include_router(proveedor_router)
```

e) Probar los endpoints

Ejecutar el servidor con `fastapi dev app/main.py` y probar en Swagger

(`http://localhost:8000/docs`). Crear archivos `.http` en `tests/` que demuestren todos los endpoints y escenarios de error.

HISTORIAS DE USUARIO

Las siguientes historias de usuario describen, desde la perspectiva del actor que opera el sistema, el comportamiento esperado de la aplicación. El actor principal es el Administrador del catálogo, único rol contemplado en esta versión. Cada historia sigue el formato "Como... quiero... para..." y se acompaña de sus criterios de aceptación, que constituyen la condición verificable de que la historia fue implementada correctamente.

ID	Historia de usuario y criterios de aceptación	Canal de verificación
----	---	-----------------------

HU-01	Registrar un proveedor Como administrador del catálogo, quiero registrar un nuevo proveedor enviando sus datos al backend, para incorporarlo a la lista de proveedores del sistema. Criterios de aceptación: • POST /proveedores crea el proveedor a partir de un cuerpo validado con Pydantic. • El código y la razón social se validan como obligatorios (min_length). • Un cuerpo inválido (código vacío o razón social con menos de 3 caracteres) es rechazado con error 422 sin alterar la lista. • Si el código ya existe, se rechaza con 409 Conflict (RN-02). • La respuesta es 201 Created con el proveedor creado (incluido su id).	Backend (REST Client)
HU-02	Listar proveedores Como administrador del catálogo, quiero listar los proveedores registrados, para tener una visión del estado de mis proveedores. Criterios de aceptación: • GET /proveedores devuelve la lista paginada (por defecto los primeros 10). • El parámetro activo permite filtrar: True solo activos, False solo inactivos, None (omitido) todos. • skip y limit controlan la paginación con validación (ge=0, le=50). • La respuesta es 200 OK con un array de proveedores.	Backend (REST Client)

HU-03	Consultar un proveedor Como administrador del catálogo, quiero consultar los datos de un proveedor por su ID, para conocer su información detallada. Criterios de aceptación: • GET /proveedores/{id} devuelve el proveedor solicitado. • Si el id no existe, devuelve 404 Not Found.	Backend (REST Client)
HU-04	Actualizar un proveedor Como administrador del catálogo, quiero actualizar los datos de un proveedor existente, para mantener su información al día. Criterios de aceptación: • PUT /proveedores/{id} reemplaza todos los campos del proveedor. • Si el id no existe, devuelve 404 Not Found. • Si el nuevo código entra en conflicto con otro proveedor, devuelve 409 Conflict (RN-02). • La respuesta es 200 OK con el proveedor actualizado.	Backend (REST Client)

HU-05	Desactivar un proveedor Como administrador del catálogo, quiero desactivar un proveedor que ya no está activo, para mantener el registro histórico sin eliminarlo. Criterios de aceptación: • PUT /proveedores/{id}/desactivar cambia el campo activo a False. • Si el id no existe, devuelve 404 Not Found. • Si el proveedor ya está desactivado, devuelve 409 Conflict (RN-05). • La respuesta es 200 OK con el proveedor actualizado.	Backend (REST Client)
---------------------	---	------------------------------

REGLAS DE NEGOCIO

Las reglas de negocio definen las restricciones que el sistema debe hacer cumplir para mantener la integridad del catálogo. Son responsabilidad del backend (capa de validación con Pydantic y lógica de los servicios). Cada regla es verificable mediante el cliente REST (.http). Las reglas se identifican con el prefijo RN.

ID	Regla de negocio	Verificación
RN-01	El código del proveedor es obligatorio (min_length=1) y no puede quedar vacío.	422 si se incumple
RN-02	El código del proveedor es único: no pueden existir dos proveedores con el mismo código.	409 Conflict si se duplica
RN-03	La razón social del proveedor es obligatoria (min_length=3).	422 si se incumple

RN-04	No se puede consultar, actualizar ni desactivar un proveedor cuyo id no exista.	404 Not Found
RN-05	No se puede desactivar un proveedor que ya está desactivado.	409 Conflict

CÓDIGOS DE ESTADO HTTP DEL API

La siguiente tabla consolida los códigos de estado que el backend debe devolver. Constituye el contrato HTTP de referencia para implementar los endpoints y para verificarlos desde los archivos .http.

Código	Significado	Cuándo se devuelve
200 OK	Operación exitosa con cuerpo	GET de listas y detalle; PUT que actualiza un recurso existente; PUT desactivar.
201 Created	Recurso creado	POST que crea un proveedor.
404 Not Found	Entidad inexistente	GET, PUT o desactivar sobre un id que no existe (RN-04).
409 Conflict	Conflicto con el estado actual	Código duplicado (RN-02) o proveedor ya desactivado (RN-05).
422 Unprocessable Entity	Datos inválidos	Cuerpo que no cumple el schema Pydantic: código vacío o razón social muy corta (RN-01, RN-03).

CONCLUSIONES ESPERADAS

Al finalizar el trabajo práctico, el estudiante debe demostrar:

- **Arquitectura modular:** El módulo Proveedores sigue el mismo patrón que Categoría y Producto: routers.py, schemas.py y services.py separados correctamente.
- **Validación Pydantic:** Los schemas validan tipos, longitudes mínimas y obligatoriedad. Datos inválidos son rechazados con 422.
- **Manejo de errores:** HTTPException con códigos HTTP adecuados (404, 409) y mensajes descriptivos en formato {"detail": "..."}.

- **Reglas de negocio:** El backend hace cumplir las reglas RN-01 a RN-05, rechazando operaciones que las violen.
- **Archivos .http:** Se entregan archivos REST Client que demuestran todos los endpoints y escenarios de error.
- **Códigos HTTP correctos:** 200, 201, 404, 409 y 422 según el contrato definido.

RÚBRICA DE EVALUACIÓN

Criterio	Qué evalúa y niveles de logro	Pts
Estructura del módulo	Completo (15): El módulo proveedor/ existe con routers.py, schemas.py y services.py correctamente estructurados, y está registrado en main.py. Parcial (8): El módulo existe pero falta algún archivo o no está registrado en main.py. Insuficiente (0): No se creó el módulo.	15
Schemas Pydantic	Completo (15): ProveedorBase, ProveedorCreate, ProveedorUpdate y ProveedorRead definidos con validaciones correctas (min_length, defaults). Parcial (8): Schemas incompletos o sin validaciones de tipo/longitud. Insuficiente (0): Sin schemas o con errores de tipo que impiden ejecutar.	15

Endpoints CRUD	Completo (25): Los 5 endpoints (POST, GET lista, GET detalle, PUT, PUT desactivar) funcionan correctamente con los códigos HTTP adecuados. Parcial (12): Faltan 1 o 2 endpoints o los códigos HTTP son incorrectos. Insuficiente (0): Menos de 3 endpoints implementados.	25
Filtros y paginación	Completo (15): GET /proveedores acepta skip, limit y activo como query parameters con validación (ge=0, ge=1, le=50). Parcial (8): Faltan parámetros o no tienen validación de tipos/rangos. Insuficiente (0): Sin filtros ni paginación implementados.	15
Reglas de negocio y errores HTTP	Completo (15): Las 5 reglas (RN-01 a RN-05) se cumplen: validación de campos obligatorios, unicidad de código, 404 en id inexistente y 409 al desactivar ya desactivado. Parcial (8): Se cumplen 3 o 4 reglas. Insuficiente (0): Menos de 3 reglas implementadas.	15
Archivos .http	Completo (15): Archivos .http que demuestran todos los endpoints y escenarios de error (éxito, 404, 409, 422). Parcial (8): Cubren solo los casos de éxito, sin escenarios de error. Insuficiente (0): Sin archivos .http.	15
TOTAL		100

MODALIDAD DE ENTREGA

- Eliminar la carpeta __pycache__ antes de comprimir.

- El proyecto completo comprimido en un único archivo .zip.
- Al descomprimir: `pip install -r requirements.txt + fastapi dev app/main.py` debe funcionar sin errores.
- Incluir los archivos .http de prueba dentro de la carpeta tests/.

RECURSOS ADICIONALES

 Documentación oficial:

FastAPI: <https://fastapi.tiangolo.com/>

Pydantic: <https://docs.pydantic.dev/>

Repositorio base:

https://github.com/profcarlosamartinez/fastapi_backend/tree/main/u1_ej_8_integrador